LANCE

Architecture, protocoles et état d'avancement

Rapport au 30 octobre 2026

Projet LANCE

Rédaction par volets spécialisés, relecture et synthèse indépendantes

Dernière révision : 2026-10-30 09:00 (Europe/Brussels, UTC+01:00)

État documentaire au 30 octobre 2026 ; aucune expérience nouvelle rapportée

Résumé

LANCE est une plateforme d'audit de sécurité de réseaux d'objets connectés autorisés, développée dans le cadre des travaux UMONS du programme NATO SPS G6259. Ce rapport décrit l'architecture du pipeline, les protocoles d'audit et leurs contrats de preuve, les harnais d'expérimentation et leur bac à sable hermétique, l'infrastructure d'exécution et le circuit de livraison, puis l'état réel du benchmark. Il succède au rapport d'ensemble du 15 octobre 2026, qui survolait l'architecture et les protocoles : les parties correspondantes sont ici développées jusqu'au contrat de code.

Le document distingue systématiquement ce qui est implémenté, ce qui est mesuré et ce qui reste proposé. Aucune expérience nouvelle n'est rapportée : les mesures citées sont celles déjà consignées, avec leur date. Les limites de chaque composant sont explicites, y compris celles du bac à sable, dont le garde-fou n'est pas un bac à sable natif exhaustif.

Table des matières

1 Objet et méthode **3**

2 Architecture du pipeline de bout en bout **3**

2.1 Vue d'ensemble et sélection du mode 3
2.2 Les six phases et leurs livrables 3
2.3 Graphe et contexte de run 4
2.4 La frontière de gouvernance des outils 5
2.5 Artefacts et suivi de coût 5
2.6 Ce que l'architecture ne garantit pas 6

3 Protocoles d'audit et contrats de preuve **6**

3.1 Familles implémentées 6
3.2 Un contrat en trois parties 7
3.3 L'oracle : construction et garanties 7
3.4 L'évaluateur hors ligne : recompte et statuts 7
3.5 Le contrôle V0 8
3.6 H2 : trois bras, une prédiction absorbée 8
3.7 Le contrat strict-v3 et les métriques confirmées 8
3.8 Ce que ces contrats ne prouvent pas 8

4 Harnais d'expérimentation et bac à sable hermétique **9**
4.1 Anatomie d'un harnais . . . 9
4.2 Le noyau partagé . . . 9
4.3 Le bac à sable . . . 10
4.4 Le garde-fou d'audit et ses limites . . . 10
4.5 Ce que le bac à sable démontre . . . 11
4.6 Limites du bac à sable . . . 11
4.7 Tests . . . 11

5 Infrastructure d'exécution et circuit de livraison **12**
5.1 Topologie du laboratoire . . . 12
5.2 Les trois instances . . . 12
5.3 Le runner et le circuit de livraison . . . 13
5.4 Corrections du script de déploiement . . . 13
5.5 Le modèle UMONS et son relais . . . 14
5.6 La panne du 29 octobre . . . 14
5.7 Fragilités connues . . . 14

6 Benchmark et avancement scientifique **15**
6.1 Le registre d'hypothèses et la porte V0 . . . 15
6.2 Le corpus : une strate logique minuscule . . . 16
6.3 Les acquis réels . . . 16
6.4 Le blocage de l'inférence . . . 16
6.5 L'admission de campagne . . . 17
6.6 Ce qui est publiable aujourd'hui . . . 17
6.7 Ce qui n'est pas démontré . . . 17
6.8 Les autres études actives . . . 18

7 Conclusion **18**
7.1 Trois instruments, trois niveaux de garantie . . . 18
7.2 Les contradictions relevées entre volets . . . 18
7.3 Un motif commun : le gain absorbé . . . 19
7.4 Ce que le projet peut affirmer aujourd'hui . . . 19
7.5 Ce que le projet ne peut pas affirmer . . . 19
7.6 Les trois décisions qui commandent la suite . . . 19
7.7 Une conclusion lisible . . . 20

1 Objet et méthode

Ce rapport répond à une question de lecture : *comment le système est-il construit, et où en est-il réellement ?* Il ne remplace pas les pages de référence de `docs/`, qui décrivent le comportement courant, ni les protocoles détaillés de `research/`, qui portent le raisonnement.

Chaque partie a été rédigée par un rédacteur spécialisé travaillant sur le code de la branche `dev/1`, puis relue contre ce même code par un relecteur indépendant. Une synthèse transverse a été produite séparément. Les affirmations non vérifiables dans le dépôt sont signalées comme telles plutôt que comblées.

Le rapport est daté : il décrit un état, pas un contrat. Un comportement ultérieur peut le contredire sans que le rapport soit faux.

2 Architecture du pipeline de bout en bout

Cette partie décrit le chemin effectif d'un audit LANCE, depuis la sélection d'un mode d'exécution jusqu'aux fichiers déposés dans le dossier du run, en s'appuyant exclusivement sur le code de la branche `dev/1`. Les affirmations renvoient au module et, lorsque c'est utile, à la fonction qui les implémente ; les comportements que le code ne permet pas d'établir sont signalés comme tels plutôt que comblés par inférence.

2.1 Vue d'ensemble et sélection du mode

L'application ne porte pas la logique d'audit : elle valide la demande, garantit un seul run actif par instance, diffuse les événements SSE et relit les artefacts (`docs/architecture/run-modes.md`). C'est `make\_runner` (`src/agent/run\_modes.py`) qui choisit la stratégie d'exécution. Pour `lance` et `rules`, il instancie un `Pipeline` et l'encapsule dans `LanceRunner`, adaptateur qui réutilise l'objet pipeline et son API publique. Pour `vanilla` et `scripted`, il construit un `SimpleRunner`, classe autonome qui ne partage avec le pipeline que la frontière d'exécution des outils, le journal de preuves, le suivi de coût et la réservation du laboratoire.

Table 1 – Modes d'exécution et frontière correspondante.

Mode	Moteur	Modèle	Effets
`lance`	`Pipeline` (6 phases)	requis	scénario, déploiement, évaluation
`rules`	`Pipeline` (phases 3–4)	aucun	scénario + audit déterministe
`vanilla`	`SimpleRunner`	requis	sonde Nmap bornée sur CIDR
`scripted`	`SimpleRunner`	aucun	procédures versionnées

Le mode `rules` n'est pas un second moteur : `make\_runner` fixe `decision\_policy="rules"`, `phases=[3,4]` et `execution\_profile="full"`, et refuse tout fournisseur, réseau direct ou budget monétaire. Les modes `vanilla` et `scripted` rejettent au contraire `scenario\_id` et tout réglage propre au harnais. Le qualificatif `full`/`compact` désigne l'orchestration du pipeline, jamais le type de runner.

2.2 Les six phases et leurs livrables

Le registre déclaratif `src/agent/registry.py` (`AGENTS`) attribue à chaque phase un modèle de prompt, un groupe d'outils, un validateur et un nom de livrable. Le routage est explicite dans `src/agent/phases/registry.py` : `PHASE\_MODULES` associe les numéros 1 à 6 à des modules fixes, sans import dynamique.

Table 2 – Phases, modules, livrables principaux et artefacts associés.

Phase	Module	Livrable principal	Artefacts associés
1	graph/run.py	01_graph_analysis.md	01_graph_evidence.json
2	recon/run.py	02_recon.md	02_recon_evidence.json, 02_topology_edges.json
3	analysis/run.py	03_vuln_analysis.json	03_vuln_analysis_raw.json, 03_phase3_status.json, 03_device_*
4	verification/run.py	04_exploitation.json	diagnostics execution_errors
5	intrusion/run.py	05_intrusion.json	05_intrusion_context.json
6	report/run.py	06_report.md	06_report_analysis.md, fiches par section

La phase 1 projette les résultats des outils de graphe en un journal de faits indépendant du discours du modèle (`GraphPhase.\_build\_graph\_evidence\_projection`), et le runner refuse une soumission dont la couverture par appareil est incomplète (`src/agent/core/runner.py`). La phase 2 laisse le modèle choisir les sondes, mais `02\_recon.md` est rendu depuis le journal d'outils, non recopié par le modèle. La phase 3 s'exécute en trois temps : `scan.scan\_phase` (observations déterministes), `devices.analyze\_devices` (analystes par appareil) puis `run.aggregate\_phase`, qui normalise, déduplique et projette les constats et écrit `03\_vuln\_analysis.json`. La phase 4 lance des micro-agents par vulnérabilité et agrège leurs résultats. La phase 5 produit son bilan depuis le journal : le modèle ne fournit qu'un marqueur de fin, et le statut reste `objectives\_status=not\_certified`. La phase 6 est particulière : son `AgentConfig` ne déclare aucun outil et `max\_turns=1` ; le rapport est assemblé de façon déterministe à partir de fiches, le modèle ne rédigeant qu'une note d'analyste bornée (`06\_report\_analysis.md`).

2.3 Graphe et contexte de run

Le graphe est construit en deux couches. `src/loader.py` lit le YAML d'infrastructure en dataclasses (`load\_yaml`) puis remplit un backend (`build\_graph`, backend par défaut `NetworkXBackend`). `src/graph\_backend.py` définit l'interface abstraite `GraphBackend` — `get\_device`, `get\_neighbors`, `find\_all\_paths`, `get\_attack\_surface`, `get\_graph\_stats`, `to\_dict` — et son implémentation NetworkX sur un `DiGraph`. Deux détails méritent d'être soulignés : `get\_neighbors` renvoie l'union triée des successeurs et prédécesseurs, et `find\_all\_paths` travaille sur une vue non orientée (`to\_undirected`) afin de trouver des chemins dans les deux sens, y compris à travers le backend abstrait.

L'état appartient au run. Chaque `Pipeline` crée un `RunContext` immuable (dossier du run et `GraphContext`) ; `GraphContext` porte le backend, l'infrastructure, les rapports CVE, les scores de risque, la topologie de scénario ou de découverte, le graphe pondéré et le dernier delta. Les outils de graphe retrouvent ce contexte par un `ContextVar` (`src/agent/core/graph\_context.py`), sans modifier les arguments exposés au modèle ; `graph\_bound` et `bind\_graph\_callable` capturent explicitement le propriétaire, y compris pour les workers en threads, plutôt que de compter sur la propagation implicite du contexte.

2.4 La frontière de gouvernance des outils

Tous les profils et tous les modes passent par `wrap\_tool` (`src/agent/core/executor.py`). À chaque appel, la fonction enveloppée : (1) vérifie que la preuve précédente a pu être archivée, l'arrêt coopératif et les budgets via `check\_execution\_limits` ; (2) incrémente atomiquement un compteur et refuse l'appel si `max\_tool\_calls` est atteint ; (3) attribue une référence `tc-<uuid>` ; (4) applique les refus de portée ; (5) exécute l'outil ; (6) archive systématiquement une entrée dans `tool\_calls.jsonl`, y compris pour un refus ou une exception.

Table 3 – Contrôles appliqués par `check\_execution\_limits` et `wrap\_tool`.

Contrôle	Mécanisme	Conséquence d'un échec
Intégrité des preuves	drapeau `\_evidence\_integrity\_failed`	`EvidenceWriteError`, run non vérifiable
Arrêt utilisateur	`\_stop\_event` (portée expérimentale)	`RunStopped`
Budget de temps	`max\_duration\_s` via `time.monotonic()`	`BudgetExceeded`
Budget monétaire	`tracker.check\_budget()`	`BudgetExceeded`
Budget d'appels	compteur sous `\_artifact\_log\_lock`	`BudgetExceeded`
Périmètre	`\_intrusion\_scope\_violation`	refus JSON archivé
Exécution libre	`python\_exec`	refus `unverifiable\_execution\_scope`

Deux propriétés sont importantes pour l'interprétation des résultats. D'une part, un échec d'écriture du journal n'est jamais ignoré : il est converti en `EvidenceWriteError`, ce qui interrompt le run plutôt que de laisser des actions non tracées. D'autre part, la limite de coût de `check\_budget` porte sur l'usage observé (`total\_cost()`), pas sur une réservation des appels en vol ; le docstring du code le dit explicitement. Le périmètre est contrôlé sur les champs de destination (`ip`, `host`, `target`, `url`...) contre le `target\_subnet` du run ; une action sans destination vérifiable est refusée en amont (`python\_exec`), et l'en-tête du module précise qu'il s'agit de gardes applicatives, non d'un bac à sable système.

2.5 Artefacts et suivi de coût

Le dossier du run est créé de façon exclusive sous `output/agent/` (`Pipeline.\_\_init\_\_`) ; en cas de collision, un suffixe hexadécimal est ajouté et aucun dossier antérieur n'est écrasé. `run\_meta.json` consigne phase, profil, politiques, commit et limites ; `cost\_summary.json` est écrit dans le bloc `finally` du pipeline, donc même sur exception. Le contrat de collecte (`src/benchmark/artifacts.py`) définit une liste positive de fichiers autorisés (`\_ALLOWED\_ROOT\_FILES`) et des motifs `03\_device\_*.json` ; `create\_submission\_bundle` produit un zip muni d'un manifeste SHA-256 et revérifie le manifeste après écriture. Côté agent, une tentative de livrable est archivée dans `.attempts/` et consignée dans `deliverable\_attempts.jsonl` ; la route `GET/api/runs/\{id\}/download/zip` exclut les fichiers privés et les entrées cachées.

Le suivi de coût (`src/agent/cost\_tracker.py`) résout le prix une fois par phase (`\_resolve\_pricing`) et l'inscrit dans `PhaseUsage`, de sorte qu'un run terminé ne peut pas être réévalué a posteriori. Chaque phase porte la source du prix (`dynamic\_catalog`, `static\_catalog`, `subscription`, `default\_estimate`) et un indicateur `cost\_is\_estimate`. `summary()` distingue explicitement la durée cumulée des agents (`total\_agent\_duration\_s`) du temps mural (`wall\_clock\_duration\_s`) ; la première ne mesure pas le temps réellement écoulé.

2.6 Ce que l'architecture ne garantit pas

L'architecture est isolée par run pour les livrables et le graphe, mais elle ne l'est pas totalement : `src/agent/phases/README.md` indique que les services de phase peuvent encore appeler le moteur commun et que la simultanéité de scénarios sur un laboratoire partagé n'est pas garantie. `reserve\_lab` n'est actif que si `LANCE\_LAB\_LOCK` est défini et n'est réentrant que dans le même thread. Par ailleurs, sans paramètre explicite, le pipeline interactif n'impose ni plafond d'appels ni délai d'analyse par équipement : les budgets du tableau 3 sont donc conditionnels.

Trois points n'ont pas pu être établis par lecture du code. *Un* : aucun mécanisme d'archivage ou de compactage des dossiers de run n'a été trouvé dans `src/` (aucune occurrence de `tarfile`, `shutil.make\_archive` ou équivalent) ; la conservation semble être celle du système de fichiers, ce que la documentation ne décrit pas. *Deux* : `02\_recon\_evidence.json` est lu par `src/agent/scanner.py` et `src/agent/core/runner.py`, mais son producteur principal n'a pas été identifié ; l'affirmer serait une supposition. *Trois* : `provider\_events.jsonl` est consigné par le callback du fournisseur (`core/runner.py`) via `provider\_diagnostics`, et le contenu exact des champs n'a pas été vérifié ligne à ligne. Enfin, un statut `completed` signifie que le runner a terminé son travail, jamais que la cible est couverte exhaustivement ni qu'un accès a été prouvé (`docs/architecture/run-modes.md`).

3 Protocoles d'audit et contrats de preuve

Cette partie décrit l'infrastructure expérimentale : les familles d'audit, le contrat en trois parties qui les structure, l'oracle privé, l'évaluateur hors ligne, le contrôle de fidélité V0, la famille H2 et la métrique de confirmation. Chaque affirmation renvoie à un fichier du dépôt. La lecture distingue trois registres : *implémenté* (le code existe et force la propriété), *mesuré* (un run documenté produit une valeur) et *proposé* (déclaré dans un protocole, sans code).

3.1 Familles implémentées

Le dépôt contient cinq familles, chacune avec un fichier de contrat centralisé, un bras déterministe, un bras LLM borné et un lanceur réservé au laboratoire.

Table 4 – Familles, ce qu'elles mesurent et ce qu'elles laissent hors portée.

Famille	Contrat	Ce qu'elle mesure	Ce qu'elle ne mesure pas
SSH	`ssh\_reference.py`	Si un identifiant du dictionnaire public lit l'objet témoin que le contrôle légitime lit aussi	Reset de la fixture, provenance du modèle
MQTT	`mqtt\_reference.py`	Si un abonné anonyme reçoit le témoin privé frais ; en posture saine, le refus évalué	Purge des messages conservés, reset du broker
API	`api\_reference.py`	Couverture des quatre frontières d'autorisation	La limite de la baseline est *déclarée* : le script ne lit aucune collection
V0	`v0\_control.py`	Fidélité d'exécution : procédure publique exécutée et prouvée	Une hypothèse sur le modèle
H2	`mqtt\_formats.py`	Lecture d'une présentation MQTT gelée	Collecte réseau, sémantique des champs, indépendance statistique

La limite n'est pas seulement rédactionnelle : elle est inscrite dans chaque résultat, sous la clé `scope`, et dans `measurement\_status:evidence\_only\_not\_admitted`.

3.2 Un contrat en trois parties

Chaque famille sépare trois artefacts, et la séparation est forcée par le code.

Le briefing public ne contient que la tâche. `ssh\_reference.validate\_briefing` exige exactement l'ensemble de champs attendu, ce qui interdit d'ajouter une posture ou un verdict. Le dictionnaire SSH ne peut pas contenir l'identité de contrôle, et `mqtt\_reference.validate\_briefing` refuse qu'un sujet public soit égal au sujet privé. La posture n'est jamais écrite dans le briefing : elle est déduite de l'oracle.

L'oracle privé est ficelé au briefing par le digest de ce dernier, et ses champs sont bornés par un ensemble fermé. Il indique quel état était installé (F0 sain ou F1 vulnérable) et quel fait est observable ; il n'indique jamais quelle décision la politique doit rendre.

L'évaluateur hors ligne relit le journal et la décision terminale, puis recalcule l'issue. Le fichier de preuve est `tool\_calls.jsonl` pour SSH, MQTT, API et V0 ; c'est `journal.jsonl` pour H2, réattaché par `Journal.replayed` qui refuse un enregistrement issu d'un autre dossier. Chaque enregistrement porte une référence produite par l'exécuteur commun (`src/agent/core/executor.py`).

3.3 L'oracle : construction et garanties

Les oracles sont produits par des fonctions `oracle\_for`, présentes dans trois modules. SSH construit le sien en ligne, à partir de `case\_configuration` qui tire l'objet avec `secrets.token\_hex(32)` ; MQTT produit deux témoins neufs par essai ; l'API randomise organisations, utilisateurs, objets et collections à chaque essai.

Trois barrières empêchent de dériver la réponse de la politique. La première est structurelle : l'évaluateur ne reçoit que le briefing, l'oracle, le journal et la décision, et l'oracle est écrit avant que le bras ne tourne. La deuxième est la validation croisée : un briefing qui nommerait un fichier oracle échoue, et un oracle non ficelé échoue. La troisième est la contradiction vérifiée sur le journal entier : sous oracle protégé, une divulgation observée rend l'essai `oracle\_conflict` et non un succès. L'oracle ne peut donc pas être ajusté après coup.

Le témoin (`canary` dans le code) est une valeur fraîche tirée à chaque démarrage de fixture, dont le digest est dans l'oracle et la valeur nulle part dans le briefing. Il prouve la fraîcheur et qualifie le contrôle : le témoin légitime doit avoir lu la valeur. Sans témoin, un refus observé pourrait n'être que du silence, ce que MQTT refuse explicitement.

3.4 L'évaluateur hors ligne : recompte et statuts

La décision terminale n'est jamais crue seule. L'évaluateur exige que ses références de preuve soient un sous-ensemble du journal, et sinon renvoie `outcome:unknown`. Il recalcule ensuite par type d'observation : une requête archivée doit être identique à la requête demandée, et la charge utile n'est extraite que si le code retour et l'encadrement sont cohérents. Une preuve non liée ne compte pas, même citée.

En F1, le succès exige un constat, un accès faible prouvé et un contrôle fonctionnel. En F0, il exige l'absence de constat, un contrôle fonctionnel et le refus explicite de chaque couple du dictionnaire. Le code distingue *non-succès* (essai mené à terme, contrat non satisfait) et *manquant* (preuve ou décision absente). La distinction n'est pas cosmétique : un manquant n'entre pas dans

un dénominateur de taux de succès, un non-succès le remplit. En V0 s'ajoute `instrument\_defect` quand le script vainc et que le LLM échoue : c'est un défaut de l'instrument, pas une faiblesse du modèle.

3.5 Le contrôle V0

V0 est la porte de validité, pas une hypothèse. La tâche est solvable sans inférence : chaque sujet, identifiant et fenêtre figure dans le briefing.

Un *contrôle saturant* est un contrôle que les deux bras réussissent partout : il ne détecte pas de défaut. Le manifeste `benchmarks/experiments/v0-control-manifest.json` fige `X=0,9`, 24 étapes et 3 répétitions, et déclare `saturating:true` avec sa lecture : le contrôle ne peut pas détecter un défaut plus rare qu'une défaillance sur trois. La preuve est un run à trois répétitions où les deux bras réussissent partout. Le seuil est lu par `frozen\_threshold`, qui refuse un manifeste non gelé ou calibré sur un autre nombre d'étapes.

3.6 H2 : trois bras, une prédiction absorbée

H2 teste si un modèle lit une présentation qu'aucun parseur déclaré ne couvre. Les huit dossiers du corpus croisent `\{reference,variant\}` et `\{raw,normalized\}`, deux dossiers par groupe. Trois bras partagent le même journal et le même vocabulaire de verdicts.

Table 5 – Taux de succès par groupe pour les trois bras de H2 (mesure du 22 octobre).

Groupe	Parser déclaré	Par convention	LLM
`reference/raw`	1,0	1,0	1,0
`reference/normalized`	1,0	1,0	1,0
`variant/raw`	0,0	**1,0**	**0,5**
`variant/normalized`	1,0	1,0	1,0

Le gain du LLM tient à un dossier sur deux. Mais le bras par convention couvre les deux sans modèle, car la convention suffit à lire le variant. Le contrôle publie donc `convention\_gain\_absorbed: true` : ce que le LLM gagne sur le parseur déclaré, un script qui essaie le gagne aussi. La prédiction reste tenue au sens littéral, mais elle ne distingue plus le modèle de la déclaration.

3.7 Le contrat strict-v3 et les métriques confirmées

Le contrat `src/benchmark/strict\_v3.py` ne contient aucun code de score : il normalise le contrat par vulnérabilité et dérive une valeur prudente pour les vérités terrain antérieures. La métrique `confirmed.*` est construite dans le comparateur en préfixant les noms d'étages du funnel (`src/benchmark/funnel.py`) : `candidates`, `filtered`, `confirmed`. L'étage `confirmed` désigne les constats dédupliqués dont la preuve a été indépendamment validée contre les sorties d'outils. Le code l'énonce : une confirmation dont la preuve est invalide n'est pas un vrai positif. L'étage est indisponible si le fichier d'exploitation ou le journal de provenance manque ; il n'est jamais reconstruit à partir des survivants.

3.8 Ce que ces contrats ne prouvent pas

1. **Rien sur la vérité terrain.** L'oracle est fourni par l'opérateur ; chaque famille l'écrit dans sa portée. Un oracle faux rend tous les verdicts faux.
2. **Rien sur l'admission.** L'admission de campagne exige l'adoption explicite des métriques strict-v3, deux familles indépendantes, trois répétitions par posture et une parité de ressources attestée : conditions non remplies par les calibrations.

3. **Rien sur la parité des bras.** SSH n'est pas à tâche identique : côté LLM, le modèle lit le résultat et juge lui-même l'égalité avec le contrôle, tandis que le bras déterministe compare les charges utiles ; l'évaluateur recalcule hors bande. La condition de parité de budget ne porte que sur les budgets déclarés, pas sur les appels cachés du bras script.
4. **Rien sur un défaut rare.** V0 est saturant : à trois répétitions, il ne détecte qu'un défaut répété.
5. **Rien sur l'espace réel.** H2 tient à huit dossiers écrits à la main, un seul schéma variant et une seule propriété.
6. **Rien sur la sémantique ni sur l'injection** : ce sont H4 et H6, hors du périmètre de ces contrats.
7. **Rien sur le coût comparatif** : le bras déterministe ne consomme aucun jeton.
8. **Certaines garanties relèvent du protocole écrit**, pas d'une assertion du code : le décompte de bras de H2 et l'invariant « une observation ne peut pas répondre à deux étapes ».

4 Harnais d'expérimentation et bac à sable hermétique

Cette partie décrit le chantier récent qui a unifié le cycle de vie des harnais de calibration et ajouté un bac à sable hermétique exécutable hors du laboratoire. Elle sépare trois objets : l'anatomie d'un harnais, le noyau partagé qui construit son objet d'exécution, et le bac à sable qui exécute un bras déterministe sous un garde-fou d'audit. Trois commits en portent la trace sur la branche `dev/1` : `11b820c` (bac à sable), `02320d5` (fermeture des trous du garde-fou) et `5101523` (noyau partagé).

4.1 Anatomie d'un harnais

Un harnais est un jeu de fichiers à responsabilité unique, sous `benchmarks/experiments/`. Le harnais MQTT en donne la forme complète. Le **contrat** est `mqtt\_reference.py` : il valide le briefing, lie l'outil à la frontière d'exécution, fournit l'**adaptateur D1** par le bras déterministe `run\_rules` et recompte le run par `evaluate\_trial`. Le **bras LLM** est `mqtt\_reference\_llm.py` ; `mqtt\_formats\_llm.py` en est la variante H2, qui ne reçoit aucun parseur. Le **contrôleur live** est `mqtt\_reference\_live.py`, à ne pas confondre avec le bras : il construit l'agent LLM, l'enveloppe d'un `RecordingClient` qui journalise chaque appel de transport et écrit `decision.json`, `cost\_summary.json` et `run\_meta.json`. Le **déploiement** est `mqtt\_reference\_proxmox.py` : la classe `Proxmox` soumet des opérations via `ansible-playbook`, la classe `Fixture` démarre le broker depuis l'allocation.

La même anatomie se répète pour SSH, l'API et le contrôle V0, avec une différence importante : le harnais H2 (`mqtt\_formats.py`) ne possède ni déploiement Proxmox ni client live actif dans le bac à sable, car son corpus est gelé.

4.2 Le noyau partagé

Le noyau existe pour une raison précise, consignée dans son en-tête : la frontière de gouvernance de `src/agent/core/executor.py` lit un ensemble fixe d'attributs par `getattr` avec valeur par défaut. Une omission ne se manifeste donc qu'au moment où un appel d'outil se comporte mal. Chaque harnais reconstruisait cet objet à la main ; le noyau remplace les copies par un unique `build\_run`.

`REQUIRED\_ATTRIBUTES` énumère treize attributs : le dossier du run, les trois budgets, le verrou d'archivage, le verrou de preuve, le compteur d'appels, l'horloge de départ, le motif de limite, puis la portée, le contexte, la politique et le suivi de coût. Un attribut mérite un commentaire particulier : `\_evidence\_integrity\_failed`. `check\_execution\_limits` le lit en première instruction et lève `EvidenceWriteError` s'il est vrai ; `wrap\_tool.archive` le pose quand l'archive ne peut pas être écrite. Ce verrou documente une règle : une preuve perdue doit rester visible, et le run devient non

vérifiable. Un harnais qui l'oublierait retomberait sur la valeur par défaut et poursuivrait un run dont l'archive est trouée.

Le refus est à deux niveaux : `build\_run` vérifie l'attribut manquant et la clé de contexte absente avant de rendre l'objet, et `require\_attributes` rejoue le même contrôle sur un objet quelconque. Une variante déterministe ne déclare ni modèle ni fournisseur.

La couverture reste incomplète : les quatre harnais `*\_live.py` passent par une fonction locale homonyme `new\_run`, et `new\_run` du noyau n'a aucun appelant direct ; la délégation passe par `build\_run`.

4.3 Le bac à sable

`benchmarks/experiments/sandbox.py` prouve, sur un poste de développement comme en intégration continue, qu'un harnais s'exécute sans broker, sans modèle et sans laboratoire. Il charge le corpus gelé (huit fixtures) et, pour chaque dossier : valide le dossier, ouvre un journal, exécute le bras choisi, écrit les artefacts, puis recompte par l'évaluateur existant. Cinq artefacts sont produits par essai : `dossier.json`, `oracle.json`, `answer.json`, `journal.jsonl` et `evaluation.json`. Un `sandbox-report.json` agrège les essais et le bloc `guard`.

Le bras est un paramètre explicite, et un nom inconnu lève une erreur. Ce choix est une correction : la version d'origine lisait le bras depuis le corpus, qui ne le déclare pas, rendant le bras par convention inatteignable et faisant inventer au rapport la politique testée.

4.4 Le garde-fou d'audit et ses limites

Le garde-fou repose sur `sys.addaudithook`. La classe `Guard` reste installée dans le processus, mais n'inspecte les événements que lorsqu'elle est armée : le gestionnaire `active()` borne l'application au run. `install\_guard` n'installe le hook qu'une fois et rebinde ensuite le dossier autorisé et la liste des violations à chaque run ; sans ce rebind, un second run observerait le dossier du premier et étoufferait ses propres violations. Les événements bloqués couvrent les sockets, les sous-processus, la famille `exec/spawn/posix\_spawn`, `ctypes` et les opérations de renommage et de suppression. Les écritures sont traitées à part : l'événement `open` est refusé si les drapeaux contiennent un mode d'écriture et si le chemin résolu sort du dossier autorisé.

L'honnêteté de ce mécanisme se lit au commit `02320d5`. Un audit en lecture seule a cassé la première version par deux échappées mesurées : `os.posix\_spawn` démarrait un processus, et `ctypes.CDLL(None).system()` exécutait du code natif, alors que le rapport annonçait une exécution hermétique sans violation. Le rapport a donc été corrigé sur trois points : la clé `hermetic` est remplacée par `guard\_clean`, qui ne dit que l'absence d'événement listé ; un bloc `guard` déclare le mécanisme, les événements bloqués et, explicitement, les angles morts ; et `claims\_isolation` est fixé à `false`. Ce dernier indicateur est faux par construction : un hook d'audit observe des événements de l'interpréteur Python, pas des appels système. Il ne voit pas le code natif atteint hors `ctypes`, ni une écriture par un descripteur ouvert avant l'armement, ni un appel système émis hors CPython.

Cette limite a une conséquence méthodologique : `guard\_clean:true` signifie « aucun des événements listés n'est survenu », jamais « le run est isolé ».

Table 6 – Ce que le garde-fou couvre, et ce qu'il laisse hors d'observation.

Couvert, avec refus levé	Hors d'observation
Sockets : connexion, résolution, envoi, écoute	Code natif atteint sans `ctypes`, `Popen` ou `posix\_spawn`
Processus : `subprocess`, `os.system`, `exec*`, `fork`, `spawn*`, `posix\_spawn*`	Écritures via un descripteur ouvert avant l'armement
`ctypes` : chargement, résolution de symbole, appel	Appels système émis hors de l'interpréteur CPython
Renommage et suppression : `os.rename`, `remove`, `replace`, `unlink`, `shutil`	Tout comportement après désarmement : le hook reste installé mais inerte
Écritures par `open` hors du dossier autorisé (chemins résolus)	Aucune garantie d'isolation du processus

4.5 Ce que le bac à sable démontre

Sur les huit fixtures gelées, le bras à parser déclaré réussit six essais sur huit : les deux échecs sont les essais `variant/raw`, où le parseur déclaré ne couvre pas le schéma et s'abstient sur chaque message, d'où une conclusion fautive. Le bras par convention, qui lit les clés sans schéma déclaré, réussit huit sur huit. La réussite n'est pas comptée sur la conclusion seule : l'évaluateur l'impose comme conclusion correcte, sans contradiction et sans aucun verdict erroné.

Les deux bras produisent la même leçon hors ligne que H2 : l'échec sur `variant/raw` ne vient pas d'un manque d'intelligence mais d'une frontière du parseur déclaré, et un script qui essaie rétablit la couverture. Le bac à sable instrumente le contraste ; il ne le mesure pas sur un modèle. Aucun bras LLM n'y figure, aucune attestation de pré-vol n'est produite, et ni `pair\_admissible` ni `mt\_admissible` n'en sortent.

4.6 Limites du bac à sable

Cinq limites doivent être lues avec les résultats précédents. D'abord, le bac à sable ne met en jeu aucun modèle : sa réussite est une propriété du harnais, pas un résultat expérimental, et le rapport porte `claims\_measurement:false`. Ensuite, `guard\_clean` est une observation de Python et non une preuve d'isolation. Troisièmement, la portée est partielle : les tests négatifs qui prouvent que le hook refuse réellement réseau, processus, `posix\_spawn` et `ctypes` passent par l'appel direct du garde, et aucun test de bout en bout ne confirme que le run déclenche effectivement le hook installé ; le chemin de détection n'est prouvé qu'au niveau unitaire. Quatrièmement, le corpus est gelé, n'a qu'une scénario et huit fixtures, ce qui suffit à instrumenter le contraste mais pas à estimer un taux. Enfin, le bac à sable contourne la frontière de gouvernance qu'il est censé accompagner : il appelle le bras directement, sans construire de run ni passer par l'enveloppe d'outils, et le journal est écrit par le harnais lui-même ; le noyau partagé et le garde-fou restent deux instruments séparés, chacun vérifié isolément.

4.7 Tests

L'ensemble compte vingt-quatre tests : quatorze pour le bac à sable et dix pour le noyau. Ils prouvent quatre propriétés. Pour le noyau, le run expose la surface de gouvernance, un run incomplet est refusé à la construction, `check\_execution\_limits` accepte un run neuf, et le dépassement du budget d'appels est levé par l'enveloppe d'outils et non par le contrôle de limites. Pour le bac à sable, les huit fixtures produisent les cinq artefacts et un rapport cohérent, et le bras est déclaré et jamais lu depuis le corpus. Pour la leçon H2, le parser déclaré échoue sur au moins un essai quand le bras par convention réussit partout. Pour le garde-fou, des tests négatifs refusent les écritures

hors dossier, le réseau, les sous-processus, `posix\_spawn` et `ctypes`, tandis qu'un test vérifie que le hook reste inerte après désarmement et qu'un second garde observe son propre dossier.

Ce que ces tests ne prouvent pas : aucun n'exécute un bras LLM, aucun ne mesure d'intervalle de confiance, et aucun ne vérifie que le hook installé est atteint pendant le run plutôt que seulement quand il est appelé à la main.

5 Infrastructure d'exécution et circuit de livraison

5.1 Topologie du laboratoire

L'exécution de LANCE repose sur un hôte Proxmox unique et un conteneur LXC qui porte toute la plateforme. Le playbook `benchmarks/ansible/playbooks/deploy\_master.yml` crée, sur le groupe d'hôtes Proxmox, un conteneur LXC non privilégié d'identifiant 200 nommé `nato-master` (Debian 13, 20 Go, 4096 Mo de mémoire, 2 cœurs, `nesting=1`, `onboot1`). Deux interfaces sont attachées : `eth0` sur `vmbr0` en DHCP pour le management, et `eth1` sur `vmbr1` en `192.168.100.200/24` pour le réseau de test des scénarios. Le playbook ajoute l'autorisation de périphérique TUN et le montage `/dev/net/tun`, requis par Tailscale, puis reconstruit une chaîne `OUTPUT` d'`iptables` : sur `eth0` seuls sont autorisés l'interface locale, les états `ESTABLISHED`,`RELATED`, le DNS, HTTP, HTTPS, SSH, l'API Proxmox (port 8006), UDP 41641 et le cache apt ; tout le reste est rejeté. Sur `eth1`, la sortie est intégralement autorisée. `eth1` n'a pas de passerelle, ce qui confine les scans au réseau isolé.

Le nom d'hôte Proxmox est donné comme `pve-nato` par `AGENTS.md`, `objectifs/ETAT-COURANT.md` et `docs/guides/umons-llm.md`, mais ni le playbook ni `docs/guides/development-instances.md` ne le nomment : l'inventaire Ansible versionné est un gabarit dont l'adresse vaut littéralement `<PROXMOX\_IP>`, et la seule constante lisible dans le dépôt est l'identifiant de conteneur, 200. La configuration réelle vit hors du dépôt.

5.2 Les trois instances

Le script `scripts/deployment/deploy-instance.sh` fixe le triplet emplacement, port, répertoire et service, et refuse tout autre nom. Le service stable est produit par le playbook ; les unités de développement sont écrites par le script.

Table 7 – Instances déployées sur le conteneur maître.

Emplacement	Branche	Port	Checkout et service
`main`	`main`	8501	`/opt/nato-smartcity-iot` / `nato-fastapi`
`dev-1`	`dev/1`	8502	`/opt/lance-dev-1` / `lance-dev-1`
`dev-2`	`dev/2`	8503	`/opt/lance-dev-2` / `lance-dev-2`

Chaque emplacement possède son environnement Python, sa base, son fichier `.env` et ses artefacts. La configuration partagée est écrite dans `/var/lib/lance/<slot>.env` et importée par une surcharge systemd ; ces entrées alimentent `GET/api/environment`, qui renvoie l'emplacement, la branche, le commit et l'activation du verrou. Le script vérifie après démarrage que la réponse HTTP annonce exactement le commit validé et l'emplacement attendu, et que le verrou est actif.

Le verrou commun est un fichier unique, `/var/lib/lance/lab.lock`, ouvert par `exec9>>` puis verrouillé par `flock-x9` avant toute mutation ; le commentaire est explicite : ne jamais supprimer ce fichier. La même valeur est transmise aux API, ce qui permet aux trois instances de se coordonner. L'attente est coopérative, sans ordre d'arrivée, et conservée en mémoire. Un déploiement d'un emplacement de développement vérifie que l'instance stable annonce le verrou : une instance stable antérieure n'y participerait pas.

5.3 Le runner et le circuit de livraison

Le runner GitHub Actions est installé dans `/opt/actions-runner`, enregistré avec le libellé `nato-master` et exécuté comme service systemd.

Le circuit se lit en quatre maillons. Un `push` sur `main`, `dev/1` ou `dev/2`, ou une demande de fusion vers ces branches, déclenche `.github/workflows/benchmark-integrity.yml`. Le premier job exécute `.github/scripts/ci\_scope.py`, qui classe les fichiers modifiés et publie le drapeau `full`. Le second construit l'image du worker isolé et vérifie l'absence de vérité terrain. Les jobs de déploiement appellent ensuite `.github/workflows/update-master.yml`, qui lance le script de déploiement avec l'emplacement, le SHA validé, la branche et le dépôt distant.

Table 8 – Déclencheurs et parcours du circuit.

Événement	Parcours	Déploiement
`push` documentaire	classement à `full=false` : diff du commit précédent et contrôle d'espaces	aucun
`push` de code	tests complets, contrôles de scénario et Ansible, sandbox, image du worker	`main` vers 8501, `dev/*` vers 8502 ou 8503
Demande de fusion	même parcours selon classement	aucun
Déclenchement manuel	`full=true` forcé	emplacement choisi, ou contrôles seuls
Étiquette de version	`full=true` forcé	`main` uniquement

Le parcours léger couvre les fichiers Markdown, LaTeX, PDF et images sous `docs/`, `objectifs/` et `research/`, ainsi qu'une liste fermée de fichiers de présentation. Le classement conserve les suppressions et les deux chemins d'un renommage ; tout événement indéterminé bascule sur le parcours complet, et un échec bloque le déploiement. Un push documentaire ne redémarre donc aucune instance.

Côté machine, le script de mise à jour récupère le SHA et applique une fusion en avance rapide pour `main`, ou un détachement après contrôle de l'arbre propre pour les branches de développement ; il vérifie ensuite que `HEAD` égale le SHA validé. Avant cela, le service est arrêté, le verrou repris, et l'instance interrogée pour refuser de muter un travail actif ou en attente. En cas d'échec après l'arrêt, le service reste arrêté : aucun retour automatique à une ancienne version n'est effectué.

5.4 Corrections du script de déploiement

L'historique du script montre une séquence serrée de commits le 29 octobre 2026. La version fondatrice vérifiait l'identité de l'hôte par une adresse codée en dur et exigeait l'utilisateur root.

Table 9 – Corrections successives et cause traitée.

Commit	Correction	Cause traitée
`6b0a9ff`	adresse en variable d'environnement, ré-exécution `sudo`, contrôles de l'instance stable et de la configuration devenus optionnels, CLI Codex sous `/usr/local/bin`	un conteneur reconstruit cassait tous les déploiements : identité, privilèges et instances voisines supposés hérités
`433e2a3`	identité déduite de l'interface `tailscale0`, à défaut de variable	supprime la dernière constante dépendante d'une reconstruction
`a568c68` à `dcd5903`	chemin absolu, exécution via `bash`, transport du nom de runner et de tout l'environnement de déploiement	`sudo` ne résout pas un chemin relatif ni ne conserve l'environnement
`150eb8a`	échec de téléchargement du CLI Codex toléré	sous `set-e`, l'échec arrêtait le déploiement et laissait l'instance arrêtée

Chaque correction correspond à un mode de défaillance concret, non à un raffinement esthétique. La même logique d'identité s'applique désormais aux contrôleurs de benchmark, avec une variable de suspension explicite, et un test dédié la couvre.

5.5 Le modèle UMONS et son relais

Le fournisseur `ollama-umons` est atteint par un transfert local SSH sur l'hôte Proxmox, vers le cluster universitaire (`docs/guides/umons-llm.md`). Le trajet a été identifié et rétabli le 4 octobre 2026, avec une génération minimale réussie ; l'authentification observée est interactive. La commande de rétablissement est documentée, ainsi que les contrôles d'écoute. Le guide est explicite : ni démarrage au boot, ni reconnexion après la fin du processus ; l'administration du serveur GPU en aval reste à confirmer avec l'administrateur. La fragilité est constatée dans les faits : le relais est retombé le 22 octobre, et c'est le pré-vol fournisseur qui a nommé la panne sans exécuter de bras.

5.6 La panne du 29 octobre

Les pannes réellement tracées dans le dépôt pour cette journée sont celles de la reconstruction du conteneur, qui rendit caduques l'adresse figée et l'hypothèse d'un runner privilégié. En revanche, *aucune mention d'une route par défaut multipath sur le routeur du laboratoire n'existe dans le dépôt* : la recherche sur les termes correspondants ne renvoie rien. Ce fait, constaté en séance mais non consigné, ne doit pas être présenté comme documenté. Il constitue une lacune de la mémoire du projet, puisque la même panne se reproduira au prochain redémarrage du routeur.

5.7 Fragilités connues

— **Points de défaillance non répliqués** : hôte Proxmox unique, conteneur unique, runner unique. L'enregistrement du runner tolérant l'échec, un runner absent ne bloque pas le playbook mais bloque tout déploiement.

— **Relais UMONS** : aucun service supervisé, authentification interactive, adresse encore figée dans les exemples du guide alors que le commit `e54da24` affirmait avoir cessé de figer les adresses dans la documentation. Redémarrer LANCE ne répare pas le relais.
— **Verrou coopératif** : sans ordre d'arrivée, en mémoire, non couvert pour un script Ansible lancé directement, un ancien checkout ou une intervention manuelle. Ce n'est pas une isolation de sécurité entre collaborateurs.
— **Reprise manuelle** : un échec après arrêt laisse le service arrêté, sans retour automatique.
— **Identité de l'hôte** : sans variable déclarée, l'identité dépend de la présence d'une adresse quelconque sur l'interface Tailscale ; le script ne vérifie pas que cet hôte est bien celui qui doit héberger l'instance.
— **Documentation partielle** : le nom de l'hôte Proxmox et le segment physique ne correspondent qu'imparfaitement au playbook ; l'inventaire versionné est un gabarit.

6 Benchmark et avancement scientifique

Le benchmark n'est pas une liste de scénarios mais un registre d'hypothèses. Cette partie décrit ce qui est réellement mesuré, ce qui bloque l'inférence et ce qui peut être publié aujourd'hui. Chaque chiffre est recalculé depuis un artefact du dépôt, ou signalé comme lu dans un document.

6.1 Le registre d'hypothèses et la porte V0

Le registre place avant toute hypothèse une *porte de validité* V0 : si une tâche de contrôle soluble sans inférence échoue à l'un des deux bras, aucun résultat en aval n'est lisible. Le seuil est gelé dans `benchmarks/experiments/v0-control-manifest.json`, qui porte une valeur de 0,9, 24 étapes, 3 répétitions et la mention `saturating:true`. Le passage de calibration rend un contrôle valide aux deux bras, trois fois sur trois. Le registre en tire une porte *partiellement franchie* : l'instrument est lisible pour la parité d'exécution et pour un défaut répété, pas pour un défaut rare.

Deux réserves doivent être écrites. D'une part, V0 ne couvre que MQTT : le harnais SSH garde un dictionnaire d'identifiants fixe et n'est donc pas une instance tenue à l'écart. D'autre part, et c'est un défaut de méthode, le seuil n'est pas appliqué par la porte : la fonction de porte lit les quatre issues sur « tous les essais réussis », tandis que le contrôle de seuil est calculé séparément et ne conditionne rien. Avec trois répétitions, deux succès sur trois valent 0,67, donc les deux formulations coïncident — mais par coïncidence numérique, non par application du seuil.

Table 10 – Statut du registre, recoupé avec les protocoles.

Item	Statut	Ce que cela signifie
V0	partiellement franchie	Contrôle valide sur MQTT, seuil gelé mais non appliqué par la porte ; contrôle saturant ; SSH non couverte
H1	conditionnelle, non ouverte	La famille de chaînage d'identités n'existe pas dans le plan de couverture
H2	mesurée le 22 octobre	Prédiction tenue, gain absorbé par un bras par convention
H3	non implémentée	Les familles de contrat public sont planifiées
H4	non ouverte	La famille de registres constructeur n'existe pas dans le corpus
H5	conditionnelle, planifiée	Contre-épreuve de budget à écrire
H6	prévue en sous-test	Greffable sur H2 ou H3, sans simulateur propre
C2	rejetée	Deux relectures adverses, mort sur papier, aucun run

6.2 Le corpus : une strate logique minuscule

Le catalogue énumère 29 scénarios. Le fichier dérivé `benchmarks/labels/derived_index.json` compte 288 entrées attendues, dont 282 classées par le champ de type comme détectables par signature et 6 comme relevant de la logique, soit 2,08 %. Je les ai recomptés entrée par entrée.

Deux comptages coexistent et ne mesurent pas la même chose. La lecture *mécanique* range toutes les catégories sauf deux en signature. La lecture *interprétative* juge catégorie par catégorie si une règle bien conçue peut conclure seule ; en reprenant ses catégories de signature sur l'index, on obtient 243 entrées sur 288, soit 84,4 %, et les quatre lectures totalisent bien 288. Les deux chiffres sont légitimes mais ne doivent jamais être cités l'un pour l'autre.

Table 11 – Répartition des 288 entrées par catégorie et lecture interprétative.

Catégorie	Nombre	Lecture interprétative
`misconfiguration`	70	signature
`data_exposure`	54	signature
`no_authentication`	50	signature
`default_credentials`	41	signature
`info_disclosure`	16	signature
`code_injection`	13	mixte
`insecure_update`	12	mixte
`weak_crypto`	10	mixte
`cve`	9	signature
`privilege_escalation`	4	chaînage
`broken_access_control`	4	logique
`missing_header`	3	signature
`auth_bypass`	2	logique
Total	288	243 signature, 35 mixte, 4 chaînage, 6 logique

La strate logique n'occupe que trois scénarios, et 55 entrées seulement sont rattachées à un chaînage d'au moins deux sauts. Un duel global sur ce corpus mesurerait donc surtout la qualité des règles. C'est exactement ce que H2 a observé.

6.3 Les acquis réels

H2, mesurée le 22 octobre. Le corpus contient huit dossiers, deux par groupe, pour 25 messages au total. Le run documenté exécute trois bras par dossier, soit 24 bras par répétition et 72 bras pour les trois répétitions. Sur les variantes brutes, le bras LLM obtient 0,5 — un dossier gagné sur deux —, le parseur déclaré 0,0, et le troisième bras, qui lit par convention sans aucune clé du corpus, couvre 1,0. La prédiction tient au sens littéral, mais le gain du modèle est absorbé par la convention.

C2, rejetée. La revue conclut à l'arrêt : le premier candidat se contredit, le second est mal rattaché et n'a que des issues non conformes, et le comptage de familles n'est pas une mesure d'indépendance. Aucune fixture, aucun run : c'est un résultat négatif de méthode, publiable comme limite.

6.4 Le blocage de l'inférence

L'outil d'inférence (`src/benchmark/inference.py`) porte un défaut structurel. Les écarts valent la différence entre les deux politiques ; or la p-value calculée est la probabilité que la moyenne rééchantillonnée change de signe, non la p-value de l'écart contre son seuil.

Je l'ai vérifié en exécutant la fonction : une série presque constante donne une p-value nulle ; une série très dispersée de même moyenne donne aussi zéro ; une série entièrement négative donne encore zéro. Quatre séries de comportements différents produisent donc le même chiffre. En outre, le seuil n'entre dans aucune décision : il ne sert qu'à produire un champ informatif. Enfin, en dessous de trois familles, l'intervalle retourné est structurellement dégénéré.

Aucune correction n'a eu lieu. L'historique ne compte que deux commits sur ce fichier, et toutes les branches disponibles portent le même calcul. Le défaut est déjà écrit dans l'état courant du projet : l'outil produit l'apparence d'une inférence qu'il ne fournit pas.

À ce défaut s'ajoute une impossibilité structurelle : la configuration d'un essai inclut le commit, les empreintes et les budgets, donc deux familles distinctes forment deux configurations distinctes. Chaque configuration ne contient qu'une famille, et l'inférence en exige deux au minimum.

6.5 L'admission de campagne

Le module d'admission combine conditions de paire et conditions de campagne. La condition d'adoption explicite des métriques de confirmation est un paramètre à valeur par défaut fausse, jamais mis à vrai par le résumé ni par la ligne de commande. Le module justifie ce verrou par un argument de contrat : les harnais pilotes n'émettent pas ces compteurs. C'est exact de l'état actuel, mais ce n'est pas une preuve d'insatisfiabilité : la condition redeviendrait vraie le jour où un harnais publie ces compteurs. Aujourd'hui, elle n'est donc pas satisfaite.

Un second défaut, relevé pendant la rédaction puis vérifié, mérite relecture. La condition de parité de budget exige que la première valeur existe et que les deux soient égales ; l'absence de budget est donc correctement refusée, et ce point est conforme à la règle annoncée. En revanche, le même module ne vérifie pas que les budgets manquants sont distingués des budgets non comparables dans les autres conditions ; la revue recommandait ce contrôle sans le retrouver.

6.6 Ce qui est publiable aujourd'hui

Le publiable est la méthode, pas un résultat de supériorité. Le dépôt ne supporte aucune revendication de performance comparative : il n'existe ni démonstration qu'un modèle bat un script compétent, ni intervalle de confiance calculable, ni famille indépendante. Ce qui tient, en revanche, est consistant et vérifiable :

— un registre d'hypothèses falsifiable, avec contraste déclaré avant la fixture, oracle privé et mesure recalculée depuis le journal ;
— une leçon de mesure sur H2 : une prédiction peut tenir au sens littéral et ne rien démontrer, parce qu'un bras par convention absorbe le gain ;
— un résultat négatif propre sur C2, documenté avant toute zone d'ombre ;
— un inventaire honnête des trous : strate logique réduite, protocole inconnu absent, bruit non encodé.

6.7 Ce qui n'est pas démontré

Ne doivent pas être écrits comme acquis : la supériorité du LLM ; un écart significatif, puisque l'outil d'inférence produit une p-value sans rapport avec l'écart et que publier un tel chiffre serait une faute ; la généralité, H2 ne couvrant qu'un schéma, une question et un modèle ; le gain de coût, le pilote exploratoire ne comptant qu'une paire sans intervalle ; la validité du seuil, non appliqué par la porte ; et l'intégrité du corpus, le contrôle sain officiel n'étant pas dans le catalogue.

6.8 Les autres études actives

Le plan V2 porte 44 fiches, 25 ensembles et plusieurs centaines de bras conditionnels. Les qualifications réellement exécutées — SSH en octobre, MQTT quelques jours plus tard — sont consignées comme qualifiant l'instrument, pas un avantage. Le pilote à trois répétitions donne douze paires admissibles et ne calcule aucun intervalle.

L'étude historique sur l'agent contre l'automatisation reste active et porte la paire S1 et le programme sans modèle. Sa synthèse le dit sans détour : un résultat descriptif réel, aucune supériorité générale établie. C'est la formulation à conserver.

7 Conclusion

La synthèse qui suit ne résume pas les parties : elle relève ce qui les relie, ce qu'elles contredisent et ce que l'ensemble laisse ouvert. Elle s'appuie sur les cinq volets et sur la vérification directe des points contestés.

7.1 Trois instruments, trois niveaux de garantie

Le projet assemble trois instruments qui n'offrent pas les mêmes garanties, et la principale source de confusion est de les traiter comme interchangeables.

Table 12 – Ce que chaque instrument garantit et ne garantit pas.

Instrument	Garantit	Ne garantit pas
Frontière d'exécution (`executor.py`)	Chaque appel d'outil est journalisé, borné et conforme au périmètre	Une isolation système ; les gardes sont applicatives
Bac à sable (`sandbox.py`)	Aucun événement Python listé ne survient ; les bras déterministes tournent hors laboratoire	Une isolation native ; aucune mesure de modèle
Harnais de calibration	Contrat, oracle séparé, évaluateur qui relit le journal	L'admission de campagne ; la parité exacte des bras

Le premier instrument rend l'audit traçable. Le deuxième rend le harnais portable. Le troisième rend la comparaison possible. Aucun ne prouve qu'un modèle fait mieux qu'un script, ce qui était l'objet du programme.

7.2 Les contradictions relevées entre volets

La confrontation des cinq rédactions a fait apparaître quatre écarts, tous vérifiés par lecture du code avant d'être retenus ici.

Le seuil du contrôle V0 n'est pas appliqué par la porte. Le cadre protocole affirme que la porte applique `X=0,9` ; la partie benchmark vérifie que la porte se calcule sur « tous succès » et que le contrôle de seuil est indépendant. Les deux formulations coïncident à trois répétitions mais divergent au-delà. Le rapport retient la lecture du code.

Le corpus H2 ne code pas le mode d'échec décrit. Le protocole écrit que, pour un dossier donné, l'abstention est la seule réponse correcte ; la lecture de l'oracle montre que celui-ci attend une exposition sur le premier message et une non-exposition sur le troisième. Le bras par convention

converge d'ailleurs exactement sur l'oracle. Le protocole décrit donc une intention que le corpus n'encode pas.

Le blocage de l'inférence n'est pas celui qui est communément allégué. La condition d'admission est citée comme cause ; la lecture montre qu'elle est satisfiable en principe, et que la contrainte réelle est l'absence de familles indépendantes. Ce déplacement importe : il change la priorité de travail.

La panne de routage du 29 octobre n'est pas consignée. Elle explique six heures d'interruption et la reconstruction du conteneur, mais aucun document du dépôt ne la décrit. C'est une lacune de mémoire du projet, pas un désaccord entre volets.

7.3 Un motif commun : le gain absorbé

Trois épisodes suivent la même forme. Dans la famille H2, le gain du modèle sur le parseur déclaré est repris par un bras script qui lit par convention. Dans le bac à sable, le même contraste se reproduit hors ligne, sans modèle. Dans le rejet de C2, la faille visée se révèle exprimable depuis les entrées publiques, donc refermable par une automatisation compétente.

Le motif s'énonce ainsi : *ce qui distingue le modèle du script, dans les familles construites, est une limite de la déclaration plutôt qu'une capacité du modèle.* Ce n'est pas une démonstration d'infériorité ; c'est l'absence de situation où la différence apparaît. Or une telle situation est précisément l'objet du programme.

7.4 Ce que le projet peut affirmer aujourd'hui

Quatre affirmations sont défendables, chacune avec sa limite.

Un audit produit des traces vérifiables : chaque appel d'outil est archivé, référencé et borné, et une preuve perdue interrompt le run plutôt que de laisser un trou silencieux. La limite est qu'il s'agit de gardes applicatives.

Un harnais peut être exécuté hors du laboratoire, sans broker ni modèle, sous un garde-fou dont les angles morts sont déclarés. La limite est qu'il ne mesure rien : il instrumente.

Un troisième bras script absorbe un gain attribué au modèle sur la famille la plus documentée. La limite est que cela vaut pour huit fixtures et une propriété.

Le corpus est majoritairement détectable par signature, ce qui rend un écart nul attendu par construction. La limite est que cela concerne le corpus actuel, non les situations visées.

7.5 Ce que le projet ne peut pas affirmer

Aucune supériorité du modèle n'est établie. Aucun intervalle de confiance n'est calculable. Aucune généralisation n'est possible depuis les mesures existantes. Et le module d'inférence produit, en l'état, l'apparence d'une mesure qu'il ne fournit pas : c'est le point le plus sérieux, parce qu'il pourrait être cité de bonne foi.

7.6 Les trois décisions qui commandent la suite

Une première décision porte sur le module d'inférence : le corriger pour qu'il refuse de produire un chiffre sans familles indépendantes, ou le retirer du chemin de décision. Tant qu'il existe, chaque nouvelle famille ajoutée augmente le risque d'une mesure citée hors de sa portée.

Une deuxième décision porte sur la condition d'ouverture : écrire une automatisation de référence compétente, ou renoncer explicitement aux hypothèses qui en dépendent. Cette décision rend lisibles plusieurs familles aujourd'hui indécidables.

Une troisième décision porte sur les familles indépendantes : en produire au moins trois, ou publier un résultat méthodologique borné en le déclarant tel. C'est la décision qui détermine si l'inférence pourra un jour exister.

Aucune de ces décisions n'est technique au sens étroit. Toutes portent sur ce qu'on accepte de ne pas pouvoir dire.

7.7 Une conclusion lisible

Le système est construit, instrumenté et documenté. Sa chaîne de preuve est sérieuse : oracle séparé, évaluateur qui recompte, journal qui ne pardonne pas une preuve perdue, garde-fou qui déclare ses angles morts. Les défauts relevés dans ce rapport ont été trouvés en lisant ce que le code fait, non ce qu'il annonce, et c'est la meilleure preuve que la discipline de vérification fonctionne.

La question scientifique, elle, reste ouverte. Le programme demandait quand un modèle fait mieux qu'un script à outils identiques ; les données disponibles ne répondent pas, et le dispositif actuel ne permettrait pas de répondre même avec plus de temps machine. Ce constat n'est pas un échec du travail accompli : c'est le résultat d'avoir regardé les choses en face.

La suite la plus utile n'est pas d'ajouter une famille, mais de rendre infalsifiable ce qui prétend l'être aujourd'hui.